

Overview

Check out this [link](#) to see the general structure of the LED strip modules.

Patterns

Here are the patterns that may (or may not) be set on the keyboard

1. Sine Wave (Breathing)

This controls the **intensity** of a chosen color.

- **Logic:** Pick a "Target Color" (e.g., Purple: $R = 150, G = 0, B = 255$).
- **The RGB Math:**

$$R_{out} = R_{target} \times \left(\frac{\sin(t) + 1}{2} \right)$$

$$G_{out} = G_{target} \times \left(\frac{\sin(t) + 1}{2} \right)$$

$$B_{out} = B_{target} \times \left(\frac{\sin(t) + 1}{2} \right)$$

- **Synchronized:** t is the same for all strips.
- **Parallel:** Use $t_{strip} = t + (strip_index \times 0.5)$ for a staggering effect.

2. The Comet (Tail Decay)

This keeps the color constant but uses the math to "dim" the pixels behind the head.

- **Logic:** The math calculates a "Brightness Factor" (F) between 0.0 and 1.0.
- **The RGB Math:**

$$F = \max\left(0, 1.0 - \frac{d \cdot |p - x|}{255}\right)$$

$$R, G, B = (R, G, B)_{base} \times F$$

- **Visual:** If your base color is Red (255, 0, 0), the head is bright red, and the tail fades to (50, 0, 0) then (0, 0, 0).

3. Rainbow Cycle (Scroll)

This is different because the math determines **which color** is shown, not how bright it is. You calculate a **Hue (H)** and convert it.

- **The RGB Math (Simplified Conversion):**
If H is your value from 0–255 from your equation:
 - **If $H < 85$:** $R = 255 - H \cdot 3, G = H \cdot 3, B = 0$
 - **If $H < 170$:** $R = 0, G = 255 - (H - 85) \cdot 3, B = (H - 85) \cdot 3$
 - **Else:** $R = (H - 170) \cdot 3, G = 0, B = 255 - (H - 170) \cdot 3$

4. Traveling Wave (Sine)

This creates a moving "pulse" of color against a background.

- **Logic:** Use the sine wave to oscillate between two colors (Color A and Color B).
- **The RGB Math:**
Define $W = \frac{\sin(\frac{2\pi}{L} \cdot \omega t) + 1}{2}$ (This gives a wave value between 0 and 1).

$$R_{out} = (R_A \cdot W) + (R_B \cdot (1 - W))$$

$$G_{out} = (G_A \cdot W) + (G_B \cdot (1 - W))$$

$$B_{out} = (B_A \cdot W) + (B_B \cdot (1 - W))$$

Based on key presses

1. The Expanding Ripple (Circular Wave)

When a key is pressed at coordinates (X_0, Y_0), a circle of light expands outward from that point.

- **The Look:** A ring of color "explodes" from the key you just hit and fades as it hits the edges of the strips.
- **The Math:** 1. Calculate Distance (d) from the pressed key (x_0, y_0) to any LED at (x, y):

$$d = \sqrt{(x - x_0)^2 + (y - y_0)^2}$$

2. Calculate Brightness (B) based on the current time since press (t_{press}):

$$B = \max(0, 255 - \text{decay} \cdot |d - (v \cdot t_{press})|)$$

(Where v is the expansion velocity)

2. The Vertical Column Flash (Synchronized)

A simpler, high-energy reaction. Pressing a key lights up the entire vertical column (one LED on each of the 4 strips) or the entire horizontal strip.

- **The Look:** "Laser" lines shooting across your desk under the keyboard.
- **The Math (Column):**

If $x_{processed} == x_{LED}$:

$$R, G, B = (R, G, B)_{max} \cdot e^{-\lambda t_{press}}$$

3. The Heat Map (Additive Interaction)

Every time you press a key, the LEDs near it get "hotter." If you type fast, the keyboard glows brighter; if you stop, it cools down.

- **The Look:** Frequent keys glow bright orange/white; unused areas stay dim blue or off.
- **The Math (Additive):**
For every press, add a value (V) to the nearest LEDs:

$$Value_{new} = \min(255, Value_{old} + \text{Impact})$$

Then, in your main loop, constantly subtract a small "Cooling" constant:

$$Value_{frame} = \max(0, Value_{old} - \text{CoolingRate})$$

4. The "Snake" Trace (Parallel/Serpentine)

The light follows the path of your typing, "flowing" from the last key pressed to the current one.

- **The Look:** A trail of light that connects your last 3–5 keystrokes.
- **The Math:** Use a **First-In-First-Out (FIFO) Buffer** to store the last 5 (x, y) coordinates. Map the brightness to the index in the buffer:

$$B(\text{key}_i) = 255 \cdot \frac{i}{\text{buffer_size}}$$

(The most recent key is 100% bright, the 5th oldest is 20% bright.)

Useful links

[ws2812 driver](#)

[pio example](#)

- [pio explanation](#)

[sdk](#)

[pio and dma](#)